

# Desenvolvimento de Aplicações Web na Empresa Zeeway

Construção do **front-end** de uma plataforma de protocolos para uma empresa de pesquisa agrícola, substituindo planilhas por registro centralizado e **análise visual** automatizada.



PROTÓCOLOS

TRATAMENTOS × REPETIÇÕES

AUTOR  
Lucas Henrique Lopes  
Costa

ORIENTADOR  
Prof. Paulo Afonso Parreira  
Júnior

CURSO  
Sistemas de  
Informação

# Roteiro da apresentação

01

## Introdução

O problema, a empresa, o cliente e o recorte do meu trabalho.

02

## Fundamentação Teórica

Dados no agronegócio, Scrum e as tecnologias adotadas.

03

## Metodologia

Como o time trabalhou e como o front-end foi arquitetado.

04

## Atividades Realizadas

**O foco desta apresentação:** os três módulos que construí, com as decisões técnicas e os desafios de cada um.

05

## Considerações Finais

Avaliação do produto, limitações, trabalhos futuros e impacto na formação.



# 01

## Introdução

Onde o estágio aconteceu, qual problema a plataforma resolve e o que exatamente foi minha responsabilidade.

## CONTEXTUALIZAÇÃO

# A pesquisa agrícola gera muito dado. E esse dado vivia em **planilhas soltas**.

Instituições de análise de solos e sementes coletam composição do solo, tratamentos aplicados, condições de safra e produtividade ao longo de anos. Registrar isso em arquivos avulsos cobra um preço.

### CONSEQUÊNCIA 01

#### Rastreabilidade frágil

Sem um registro único, é difícil saber quem alterou o quê e quando.

### CONSEQUÊNCIA 02

#### Regras não aplicadas

A planilha aceita qualquer valor. As regras do domínio ficam na cabeça das pessoas.

### CONSEQUÊNCIA 03

#### Análise manual e lenta

Cada gráfico entregue ao cliente final precisava ser montado à mão.

— ONDE O ESTÁGIO ACONTECEU

# Zeeway

Empresa lavrense de tecnologia, fundada no ecossistema de inovação da **UFLA**, que desenvolve software e entrega soluções personalizadas para seus clientes.

15 a 30

colaboradores diretos

≈ 40

clientes diretos

≈ 500

clientes indiretos

## O CLIENTE DO PROJETO

### Empresa de pesquisa agrícola de Lavras (MG)

Atua no agronegócio, especializada em **análises de solos e sementes**.

## O PEDIDO

### Uma plataforma própria de protocolos

Registro centralizado de safra, componentes do solo e produtividade, com acesso distribuído aos colaboradores.



# O que é um protocolo

É o registro completo de um experimento agrícola. Tudo na plataforma orbita em torno dele.

## IDENTIFICAÇÃO

### Código, tipo e cliente

Tipo Ret ou Comercial, classes do experimento e status atual.

## DELINEAMENTO

### Tratamentos e repetições

Quantos tratamentos serão comparados e quantas vezes cada um se repete no campo.

## CAMPO

### Local, solo e manejo

Cultura, variedade, datas de plantio e colheita, adubação, espaçamento e parcela.

## RESULTADO

### Avaliações por momento

Medições coletadas ao longo da safra, com fotografias e campos calculados.



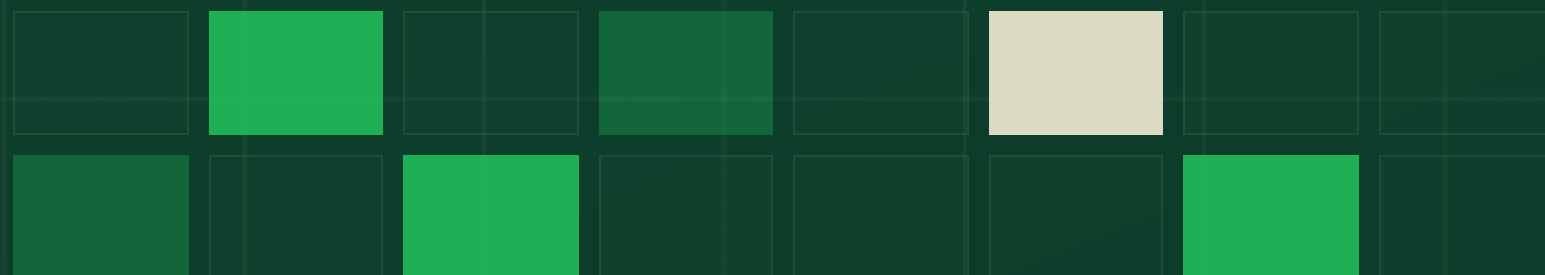
# O que este trabalho se propôs a fazer

## OBJETIVO DO ESTÁGIO

01

### Aplicar a graduação em ambiente corporativo real

Vivenciar a rotina de desenvolvimento de software em equipe e os desafios da engenharia de requisitos fora da sala de aula.



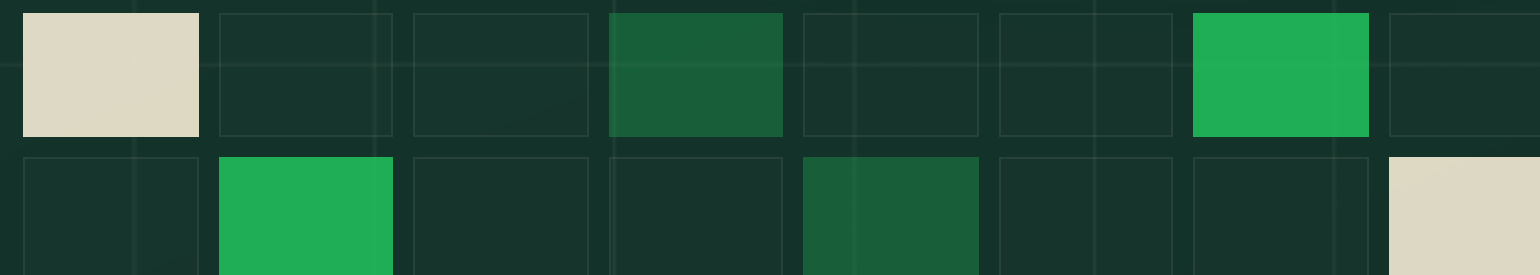
- Equipe estruturada, com papéis e cerimônias definidos
- Cliente externo e regras de negócio que ninguém tinha escrito
- Entregas incrementais validadas a cada Sprint

## OBJETIVO DO RELATÓRIO

02

### Documentar de forma estruturada o que foi construído

Registrar as decisões de projeto com a justificativa técnica de cada uma e refletir criticamente sobre o processo.



- A arquitetura front-end adotada e as tecnologias
- A metodologia de trabalho e a coordenação entre as equipes
- Os desafios técnicos e o impacto na formação

# Minha responsabilidade foi o **front-end**

O back-end da plataforma ficou a cargo de outra equipe da Zeeway, dedicada ao servidor, à modelagem do banco e às regras de negócio do lado do servidor.

## SOB MINHA RESPONSABILIDADE

### Camada de interface e experiência do usuário

Telas, formulários, tabelas, estado da aplicação, consumo da API, autenticação no cliente, controle de acesso por rota e toda a geração de gráficos.

## OUTRA EQUIPE DA ZEEWAY

### API REST, banco de dados e regras no servidor

Persistência, autenticação no servidor, consolidação dos dados dos dashboards e emissão dos tokens JWT.



02

# Fundamentação Teórica

Os conceitos e as tecnologias que sustentaram as decisões tomadas no projeto.

# Agricultura 4.0: o dado como insumo de decisão

Sensores, IoT, computação em nuvem e análise de dados passam a orientar a decisão no campo. Plataformas web que centralizam registro e análise são a evolução natural do uso descentralizado de planilhas.

## ANTES

### Planilha eletrônica

- Arquivo por pessoa, versão por e-mail
- Validação depende de disciplina humana
- Gráfico montado manualmente a cada relatório

## DEPOIS

### Plataforma web própria

- Registro único, acessível por perfil
- Regras do domínio aplicadas pelo sistema
- Gráficos gerados automaticamente a partir do dado

# Scrum: iterativo e incremental

A cada **Sprint**, de duração fixa, uma nova versão do sistema com novas funcionalidades é entregue.

|                                                                                                                               |                                                                                    |                                                                                                                              |                                                                                  |                                                                                                                               |  |
|-------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|--|
| <div><div>PAPEL</div><div>Product Owner</div><div>Prioriza o Product Backlog e define o que entra em cada Sprint.</div></div> |                                                                                    | <div><div>PAPEL</div><div>Scrum Master</div><div>Garante a aplicação correta do framework e remove impedimentos.</div></div> |                                                                                  | <div><div>PAPEL</div><div>Time de desenvolvimento</div><div>Executa as tarefas planejadas e entrega o incremento.</div></div> |  |
| <div><div>CERIMÔNIA</div><div>Planning</div><div>Escopo da Sprint</div></div>                                                 | <div><div>CERIMÔNIA</div><div>Daily</div><div>Progresso e impedimentos</div></div> | <div><div>EXECUÇÃO</div><div>Sprint</div><div>Duração fixa</div></div>                                                       | <div><div>CERIMÔNIA</div><div>Review</div><div>Incremento ao cliente</div></div> | <div><div>CERIMÔNIA</div><div>Retrospective</div><div>Melhoria do processo</div></div>                                        |  |

# A stack, organizada por responsabilidade

## APRESENTAÇÃO

### React + TypeScript

Componentização e tipagem estática. Material UI com tema próprio do cliente, TanStack Table para tabelas e ApexCharts para gráficos.

## ESTADO

### Zustand + Immer

Estado global leve, com middleware de persistência para manter a sessão. Formik e Yup para o estado e a validação dos formulários.

## DADOS

### Axios + React Query

Cliente HTTP com injeção de token e camada de cache e sincronização que evita requisições desnecessárias.

## BUILD

### Vite

Servidor de desenvolvimento com Hot Module Replacement, refletindo alterações no navegador instantaneamente.

## BIBLIOTECAS DE APOIO, CADA UMA RESOLVENDO UM PROBLEMA ESPECÍFICO

jwt-decode · leitura do token

mathjs · campos calculados por fórmula

JSZip · fotos de campo em arquivo único

use-react-screenshot · exportação em PNG

qrcode · identificação em campo



# 03

## **Metodologia e Processo**

Como o time trabalhou, como os requisitos foram levantados e como o front-end foi arquitetado.



# Sprints de duas semanas

- Na **planning**, as tarefas priorizadas do Product Backlog eram detalhadas no Sprint Backlog
- Cada item tinha identificador, estado, título, área de produto e marcadores no **Azure DevOps Boards**
- Nas **dailies**, cada um relatava progresso, plano do dia e impedimentos
- Na **review**, o incremento era apresentado ao cliente. Na **retrospective**, o time revia o processo

QUADRO DE TAREFAS

| A FAZER | EM ANDAMENTO | EM REVISÃO | CONCLUÍDO |
|---------|--------------|------------|-----------|
| #412    | #398         | #377       | #351      |
| #415    | #402         |            | #358      |
| #419    |              |            | #366      |

O fluxo permitiu **entregas incrementais** e validação contínua das funcionalidades, reduzindo o risco de retrabalho.



# As regras do domínio não estavam escritas em lugar nenhum

Como o cliente atua em pesquisa agrícola, boa parte das regras era específica do domínio e não estava formalmente documentada. Isso exigiu reuniões frequentes para esclarecer o comportamento esperado.

**Tipo do protocolo**

RET OU COMERCIAL

×

**Cultura selecionada**

DEFINIDA NO PROTOCOLO

×

**Classe do experimento**

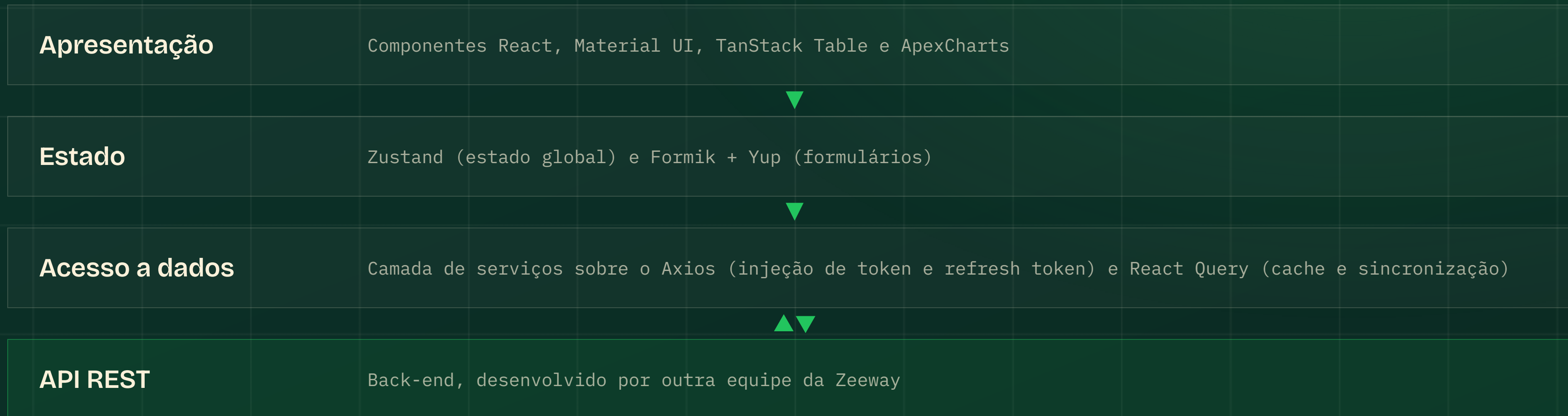
UMA OU MAIS CLASSES



**Obrigatoriedade e formato de cada campo do formulário**

- Nenhuma dessas combinações estava descrita em documento
- Cada variação precisava ser esclarecida diretamente com a equipe do cliente
- A regra acordada virava esquema de validação, refinado a cada **Sprint**

# Três camadas, três responsabilidades



A separação acomodou o volume de telas e a complexidade do domínio: cada camada tem um motivo único para mudar.

# A estrutura de diretórios reflete as camadas

```
src/  
  components/  tabelas, modais, graficos  
  config/      constantes globais  
  contexts/    contextos de React  
  hooks/       useForm, useRoles, React Query  
  layouts/     autenticao e listagem  
  pages/       Login, Protocols, Dashboard...  
  routes/      definicao e guarda de rotas  
  services/    chamadas a API (Axios)  
  store/       estado global (Zustand)  
  theme/       tema do Material UI  
  types/       tipos compartilhados  
  utils/       axios, arquivos, debounce
```

- **components, pages e layouts** concentram a parte visual da aplicação
- **store, services e routes** tratam do estado global, da comunicação com a API e da definição das rotas
- **hooks** centraliza os hooks customizados, como **useForm** e **useRoles**
- **utils** reúne funções auxiliares reutilizadas em contextos diferentes

# Como front e back andaram em paralelo

A estratégia foi paralelizar o trabalho a partir de uma interface comum, acordada antes de qualquer código.

| ETAPA 01                                                                  | ETAPA 02                                                                             | ETAPA 03                                                            | ETAPA 04                                                                  |
|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------|---------------------------------------------------------------------------|
| <b>Contrato acordado</b>                                                  | <b>Trabalho simultâneo</b>                                                           | <b>Integração</b>                                                   | <b>Ajuste</b>                                                             |
| Caminho do endpoint, método HTTP e estrutura dos dados de entrada e saída | Back implementa o endpoint. Front constrói a tela consumindo dados mascarados (mock) | Com o endpoint disponível, troca-se a fonte dos dados pela API real | Mudanças pontuais no contrato negociadas na daily ou na planning seguinte |

Nenhuma das equipes ficava permanentemente em ritmo diferente da outra.



# 04

## **Atividades Realizadas**

A plataforma de protocolos, módulo a módulo: o que foi construído, as decisões técnicas por trás e os desafios enfrentados em cada etapa.



# Três módulos, selecionados por relevância e complexidade técnica

## MÓDULO 01

### Autenticação e Controle de Acesso

Ponto de entrada da plataforma. Sessão com JWT, renovação transparente do token e restrição de funcionalidades por perfil de usuário.

## MÓDULO 02

### Registros Agrícolas

O núcleo do sistema. Listagem, cadastro e edição dos protocolos, com formulários extensos, estruturas dinâmicas e validação declarativa.

## MÓDULO 03

### Análise e Dashboards

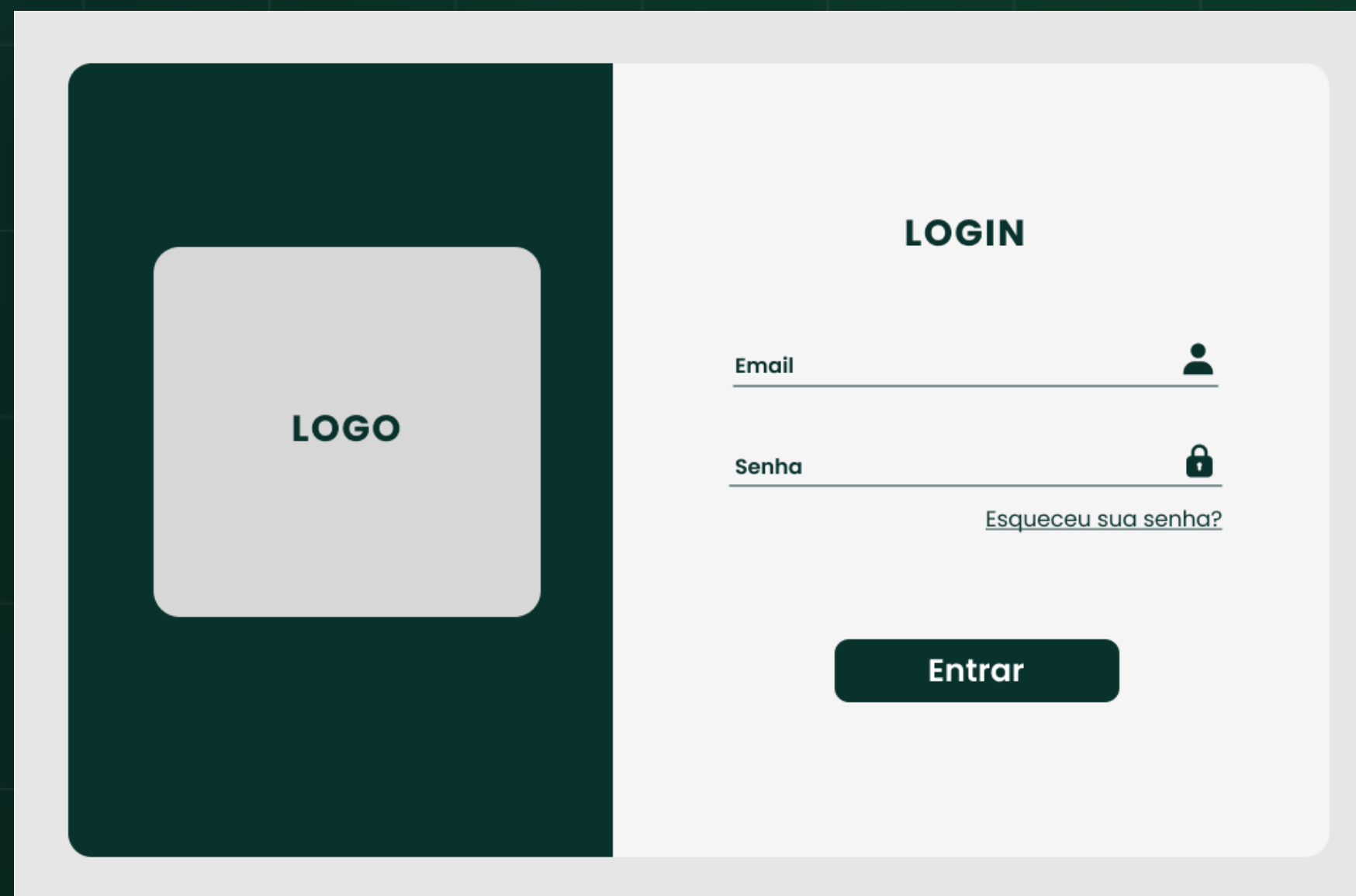
O diferencial em relação às planilhas. Geração automatizada de gráficos e exportação com a identidade visual da empresa.



## MÓDULO 01 · AUTENTICAÇÃO

# O ponto de entrada de todos os usuários

- Tela construída seguindo a **identidade visual** da empresa cliente
- Campos de e-mail e senha, com recuperação de senha
- Após o login, a API devolve um par de **tokens JWT**: access token e refresh token
- A sessão é persistida no armazenamento local, mantendo o usuário conectado entre acessos



TELA DE LOGIN DA PLATAFORMA

# Do token à sessão: Zustand, Immer e jwt-decode

- O store é criado com **persist**, que salva o conteúdo em armazenamento local sob o nome **UserData**
- **produce**, do Immer, expõe um rascunho mutável do estado e permite escrever as alterações de forma direta
- O access token é decodificado e nome, identificador, e-mail e perfil são extraídos das suas claims
- O perfil extraído aqui é o que governa todo o controle de acesso da aplicação

```
export const loginStorePersist = create(
  persist((set, get) => ({
    // ...estado inicial omitido
    setLoginInfo: ({ data }) =>
      set(produce((draft) => {
        draft.isAuthenticated = true;
        draft.session = {
          accessToken: data.data.accessToken,
          refreshToken: data.data.refreshToken,
        };
        const decoded = jwtDecode(data.data.accessToken);
        draft.userData = {
          name: decoded.unique_name,
          id: decoded.nameid,
          email: decoded.email,
          roleId: decoded.role,
        };
      })),
  })), { name: "UserData" })
);
```

# Renovação de sessão **transparente** para o usuário

- Todas as chamadas passam por uma função de serviço que anexa o token ao cabeçalho de autorização, sem repetir essa lógica em cada requisição
- Um interceptador de resposta age apenas quando o código é **401** e a requisição ainda não foi reexecutada
- Nesse caso, o refresh token é usado para obter um novo access token e a requisição original é **reenviada**
- O usuário não vê nada: não há logout inesperado no meio do preenchimento de um protocolo

```
axiosInstance.interceptors.response.use(
  (response) => response,
  async (error) => {
    const originalRequest = error.config;
    const status = error.response?.status;
    if (status === 401 && !originalRequest._retry) {
      const { refreshToken } =
        loginStorePersist.getState().session;
      originalRequest._retry = true;
      const response = await axiosInstance.post(
        "/user/refreshToken", { refreshToken }
      );
      const newToken = response.data.data.accessToken;
      loginStorePersist.getState()
        .setLoginInfo({ data: response.data });
      originalRequest.headers["Authorization"] =
        `Bearer ${newToken}`;
      return axiosInstance(originalRequest);
    }
    return Promise.reject(error);
  }
);
```

# Controle de acesso por perfil, no nível da rota

## PERFIL

### Administrador

Acesso completo, incluindo a gestão de usuários.

## PERFIL

### Técnico

Acesso restrito ao registro e à consulta de protocolos.

- Um componente guardião envolve as rotas e decide, antes de renderizar, se o usuário pode estar ali
- Sem autenticação, redireciona para o login. Sem permissão de administrador em rota restrita, redireciona para a raiz

```
const ProtectedRouteGuard = ({ children, adminRoute }) => {  
  const { isAuthenticated, userData: { roleId } } =  
    loginStorePersist((state) => state);  
  const { isUserRole } = useRoles();  
  
  if (!isAuthenticated) {  
    return <Navigate to="/login" replace />;  
  }  
  if (adminRoute && !isUserRole("administrador", roleId)) {  
    return <Navigate to="/" replace />;  
  }  
  return children;  
};
```

LOGO

### Usuários

Pesquisar

Filtrar

Adicionar

| Nome                       | Tipo | Email                 | Contato          |  |
|----------------------------|------|-----------------------|------------------|--|
| Nome Sobrenome e Sobrenome | Adm  | maria.maria@email.com | (00) 0 0000-0000 |  |
| Nome Sobrenome e Sobrenome | Tec  | maria.maria@email.com | (00) 0 0000-0000 |  |
| Nome Sobrenome e Sobrenome | Tipo | maria.maria@email.com | (00) 0 0000-0000 |  |
| Nome Sobrenome e Sobrenome | Tipo | maria.maria@email.com | (00) 0 0000-0000 |  |
| Nome Sobrenome e Sobrenome | Tipo | maria.maria@email.com | (00) 0 0000-0000 |  |
| Nome Sobrenome e Sobrenome | Tipo | maria.maria@email.com | (00) 0 0000-0000 |  |
| Nome Sobrenome e Sobrenome | Tipo | maria.maria@email.com | (00) 0 0000-0000 |  |
| Nome Sobrenome e Sobrenome | Tipo | maria.maria@email.com | (00) 0 0000-0000 |  |

« 1 2 3 4 5 »

GESTÃO DE USUÁRIOS, ACESSÍVEL APENAS PELO ADMINISTRADOR

MÓDULO 01 · AUTENTICAÇÃO

# Gestão de usuários

- Visualizar, adicionar, editar e excluir os colaboradores cadastrados
- Listagem com nome, perfil, e-mail e contato, com **busca textual** e filtro por tipo de perfil
- Criação e exclusão em **modais**, para agir sem perder o contexto da página
- Formulários com **Formik** para o estado dos campos e **Yup** para a validação antes do envio



# Listagem de protocolos com TanStack React Table

| CÓDIGO              | TIPO      | CLIENTE    | CLASSE        | CULTURA | STATUS       |
|---------------------|-----------|------------|---------------|---------|--------------|
| ADAMT_V1_SAFRA23/24 | Ret       | Cliente 01 | Fertilizantes | Milho   | Concluído    |
| BXQRT_V2_SAFRA23/24 | Comercial | Cliente 02 | Nutrição      | Soja    | Em andamento |
| CLMPT_V1_SAFRA24/25 | Ret       | Cliente 03 | Milharal      | Café    | Não iniciado |

|                                                                                                                                      |                                                                                                               |                                                                                                                                         |                                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <div>RECURSO</div> <div>Ordenação e paginação</div> <div>Recursos avançados de tabela sobre um volume crescente de protocolos.</div> | <div>RECURSO</div> <div>Filtragem</div> <div>Busca por código, tipo, cliente, classe, cultura e status.</div> | <div>RECURSO</div> <div>QR Code</div> <div>Identificador único que, escaneado <b>em campo</b>, abre direto a página do protocolo.</div> | <div>RECURSO</div> <div>Exportação</div> <div>Saída para planilha, preservando o fluxo já conhecido pelo cliente.</div> |
|--------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|



# O maior desafio técnico do projeto

Um único protocolo reúne dezenas de campos. Para não transformar isso em um formulário infinito, ele foi organizado em **oito abas**.

|             |            |             |       |      |            |        |           |
|-------------|------------|-------------|-------|------|------------|--------|-----------|
| Informações | Avaliações | Tratamentos | Local | Solo | Aplicações | Manejo | Dashboard |
|-------------|------------|-------------|-------|------|------------|--------|-----------|

CAMPOS OBRIGATÓRIOS

O mínimo para um protocolo existir

Tipo (Ret ou Comercial), código, cliente, classes do experimento, status, número de tratamentos e número de repetições.

CAMPOS OPCIONAIS

Preenchidos conforme a pesquisa avança

Cultura, variedade, cultura anterior, tipo de plantio, datas de plantio, colheita e emergência, adubação, espaçamento, delineamento e parcela útil.

Voltar

Altura

⊕ Adicionar nova coluna

1º Momento de Avaliação

✎

editar tabela

|     | PL1         | PL2         | PL3         |
|-----|-------------|-------------|-------------|
| 101 | 10          | 2           | 5           |
| 201 | Lorem ipsum | Lorem ipsum | Lorem ipsum |
| 102 | Lorem ipsum | Lorem ipsum | Lorem ipsum |
| 202 | Lorem ipsum | Lorem ipsum | Lorem ipsum |

⊕ Adicionar foto

<



nome da foto



nome da foto



nome nome nome da foto

>

2ª Momento de Avaliação

✎

editar tabela

|     | Lorem ipsum | Lorem ipsum | Lorem ipsum |
|-----|-------------|-------------|-------------|
| 101 | Lorem ipsum | Lorem ipsum | Lorem ipsum |
| 102 | Lorem ipsum | Lorem ipsum | Lorem ipsum |
| 103 | Lorem ipsum | Lorem ipsum | Lorem ipsum |
| 104 | Lorem ipsum | Lorem ipsum | Lorem ipsum |

⊕ Adicionar foto

⬇

<



nome da foto



nome da foto



nome nome nome da foto

>

ABA DE AVALIAÇÕES, COM TABELAS POR MOMENTO E FOTOGRAFIAS DE CAMPO

MÓDULO 02 · REGISTROS AGRÍCOLAS

# Uma estrutura que muda de forma

- O número de **colunas** da tabela varia conforme o número de repetições configurado na aba Informações
- Novas linhas podem ser adicionadas conforme a necessidade da avaliação
- Alguns campos são **fórmulas**, expressões calculadas a partir de outros campos, avaliadas em tempo de execução com **mathjs**
- Cada momento de avaliação aceita **fotografias de campo** vinculadas ao registro

# Todas as fotos de uma avaliação em um único arquivo

- O problema: baixar dezenas de fotografias uma a uma inviabilizava o trabalho posterior do pesquisador
- As imagens são buscadas na API por uma **mutation do React Query** e convertidas em blobs
- As promessas são aguardadas **em paralelo** com **Promise.all**
- O **JSZip** agrupa tudo em memória e o arquivo .zip é entregue ao usuário

```
const handleDownloadImages = async (evaluation) => {
  getImages(evaluation.id, {
    onSuccess: async ({ data }) => {
      const images = data.data;
      const blobs = await Promise.all(
        images.map((img) =>
          fetch(img.url).then((res) => res.blob())
        )
      );
      const zip = new JSZip();
      blobs.forEach((blob, i) =>
        zip.file(`${images[i].title}.jpg`, blob)
      );
      const zipFile = await zip.generateAsync({
        type: "blob",
      });
      saveFile(zipFile, evaluation.name);
    },
  });
};
```

# Traduzir a regra do agrônomo em **esquema de validação**

- A maior dificuldade do módulo foi traduzir as regras do cliente para a lógica do formulário, exigindo validação contínua a cada Sprint
- Campos numéricos combinam **required** com **nullable**: podem ficar nulos durante o preenchimento parcial, mas são exigidos no envio
- `array().min(1)` garante ao menos uma classe selecionada
- A mensagem de cada regra é o texto exibido ao usuário quando ela é violada

```
const validationSchema = Yup.object().shape({
  isRET: Yup.bool().required("0 campo e obrigatorio"),
  code: Yup.string().required("0 campo e obrigatorio"),
  culture: Yup.string(),
  experimentalDesign: Yup.string().nullable(),
  numberOfTreatments: Yup.number()
    .required("0 campo e obrigatorio").nullable(),
  numberOfRepeats: Yup.number()
    .required("0 campo e obrigatorio").nullable(),
  plantingDate: Yup.string().nullable(),
  harvestDate: Yup.string().nullable(),
  classes: Yup.array().min(1, "0 campo e obrigatorio"),
  clientId: Yup.string().required("0 campo e obrigatorio"),
});
```

LOGO

Orçamento / Informações

Informações

Administrativo

\*Tipo:

☐Ret

☐Comercial

☐Vitrine

\*Código do protocolo

ADAMT\_VI\_SAFRA23/24

Sequência protocolo

Lorem ipsum

Data de envio da proposta

-- / -- / --

Versão

VI

Localidade

Campo

Cliente

Selecione

Responsável interno

Selecione

Valor

R\$

Sigla

Cidade

Contato

Nome

Telefone

Email

Informações

Cultura

\*Classe

Selecione

Safra

Selecione

\*Nº de tratamentos

\*Nº de repetições

Área total (m²)

MÓDULO DE ORÇAMENTOS, COM O MESMO PADRÃO DE FORMULÁRIO

MÓDULO 02 · REGISTROS AGRÍCOLAS

# O padrão se pagou em outros módulos

- A mesma organização foi reaproveitada no módulo de **orçamentos**: estrutura em abas, agrupamento por área e separação visual entre seções
- Os componentes de seleção, campos de data e modais foram reutilizados em contextos diferentes
- O hook customizado **useForm** padronizou o controle de valores e a orquestração das validações em todas as telas
- Isso importava porque um protocolo podia conter dezenas de tratamentos, cada um com múltiplas repetições e avaliações



**Aqui está o que a planilha **não fazia:**  
transformar dado bruto em gráfico, sem  
ninguém montar nada.**



# Do dado de avaliação à imagem exportada

| ETAPA 01                                                          | ETAPA 02                                                                      | ETAPA 03                                                                    | ETAPA 04                                                     | ETAPA 05                                                             |
|-------------------------------------------------------------------|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------|--------------------------------------------------------------|----------------------------------------------------------------------|
| <b>Dados do back-end</b><br>Valores já consolidados pelo servidor | <b>Mapeamento para séries</b><br>Vetores de rótulos (xAxis) e valores (yAxis) | <b>Opções do ApexCharts</b><br>Tipo, cores por categoria e rótulos de dados | <b>Renderização</b><br>react-apexcharts, um gráfico por card | <b>Exportação PNG</b><br>use-react-screenshot, com identidade visual |

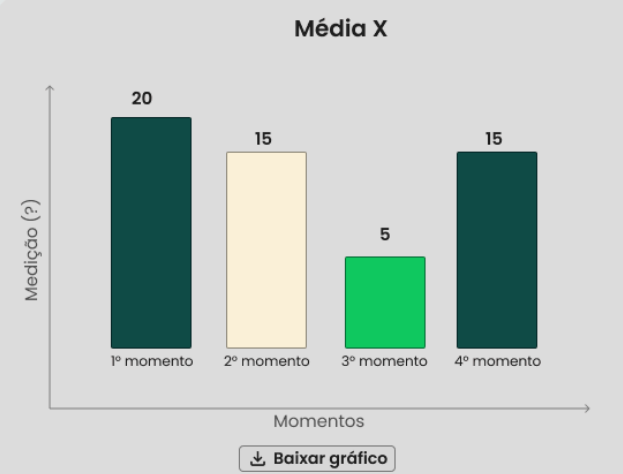
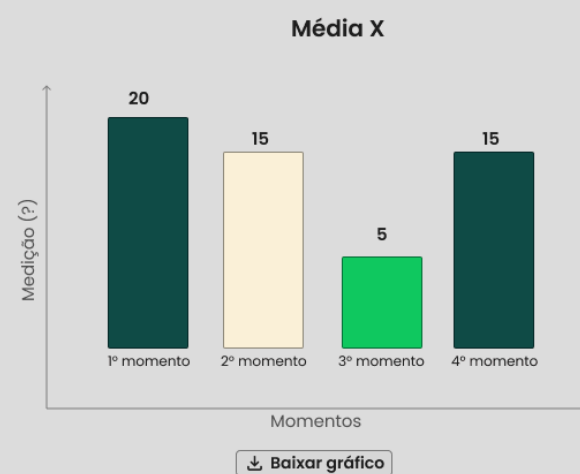
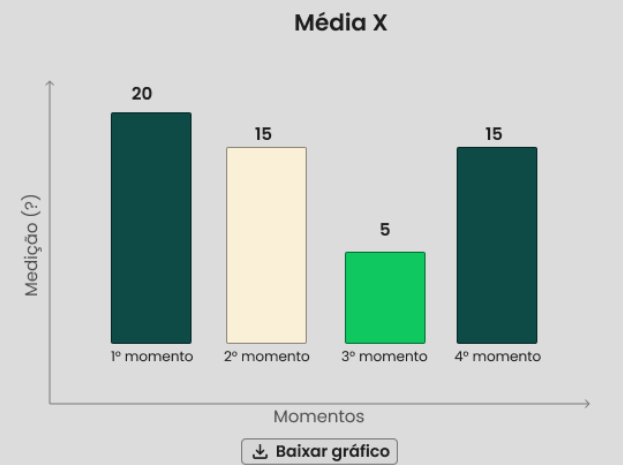
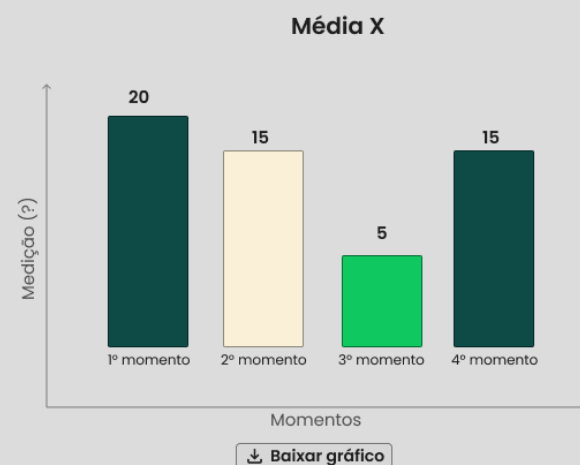
```
const series = [{ name: xAxisTitle, data: yAxis }];

const options: ApexOptions = {
  chart: { type: "bar", toolbar: { show: false } },
  plotOptions: { bar: { distributed: true } },
  dataLabels: { enabled: true },
  xaxis: { categories: xAxis },
  yaxis: { max: maxValue + 1 },
};
```

← Voltar

## Altura

Baixar gráficos



## MÓDULO 03 · ANÁLISE E DASHBOARDS

# Nenhum ajuste manual

- Os componentes recebem os dados já consolidados e montam **sozinhos** as séries e as opções do ApexCharts
- Barras verticais e horizontais comparam resultados entre tratamentos, categorias e momentos de avaliação
- Cada gráfico ocupa um **card próprio**, com opção de download individual
- Colunas coloridas por tratamento e valores numéricos sobre elas, para comparação imediata

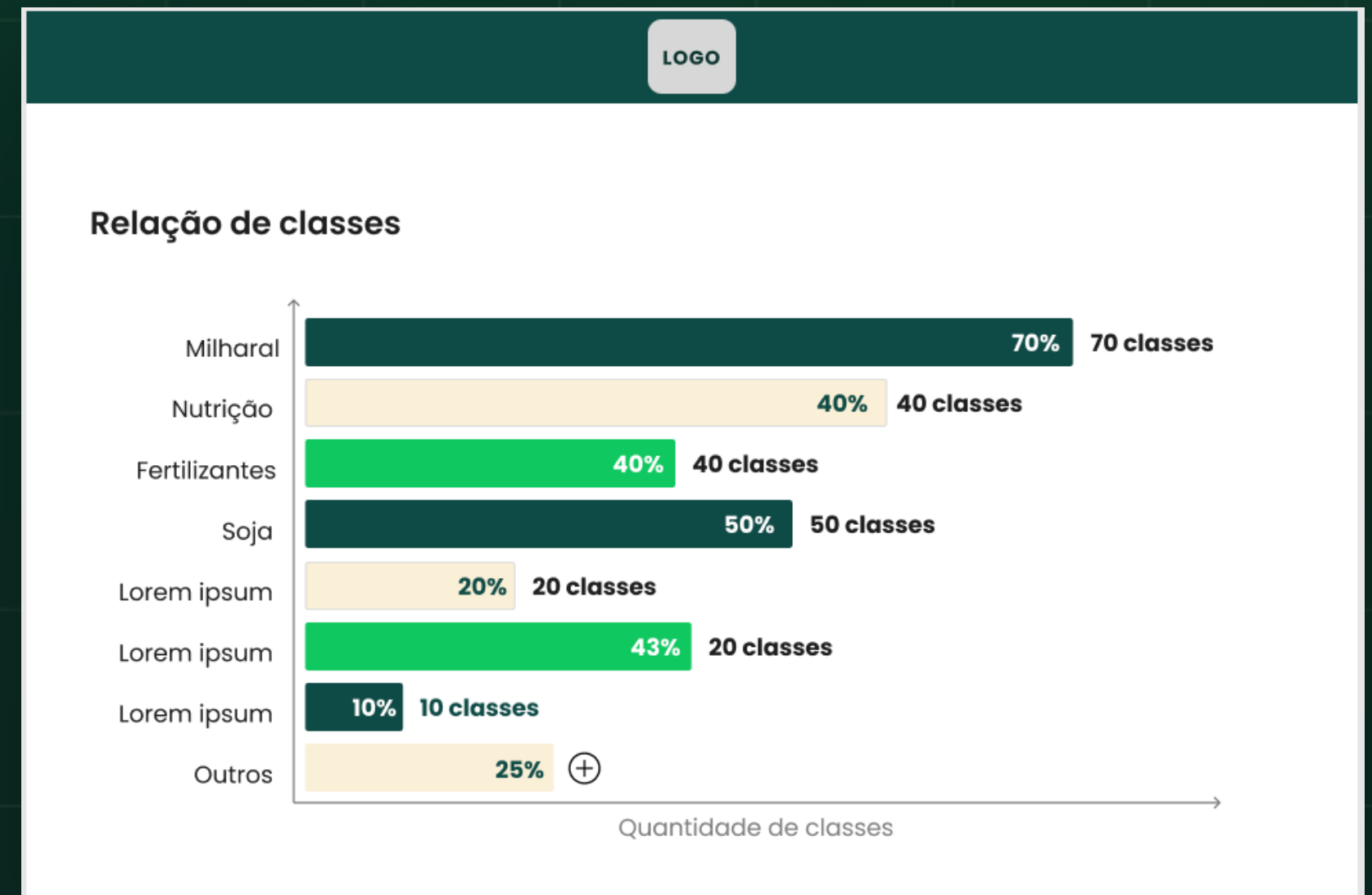
GRÁFICOS GERADOS AUTOMATICAMENTE PARA UMA AVALIAÇÃO

## Pronto para o relatório do cliente

- Um elemento oculto na tela mantém o gráfico acompanhado do **logotipo** e do título dos dados
- A captura desse elemento é feita com **use-react-screenshot**, que devolve a imagem em base64
- Um link é criado dinamicamente e o clique programático força o download

```
const [_, takeScreenshot] = useScreenshot();

const downloadChart = async (format) => {
  const imageBase64 =
    await takeScreenshot(imageRef.current);
  const link = document.createElement("a");
  link.href = imageBase64;
  link.download =
    `${imageTitle} - ${chartTitle}.${format}`;
  link.click();
};
```



PNG EXPORTADO, COM LOGOTIPO E TÍTULO DOS DADOS

LOGO



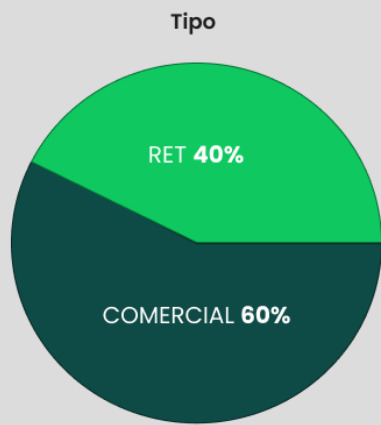
## Dashboard / Protocolos

Protocolos

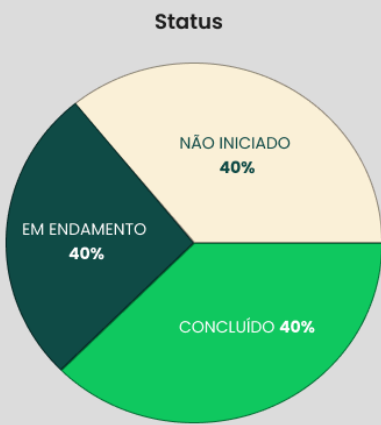
Financeiro

Baixar gráficos

Dados gerais dos protocolos ▾

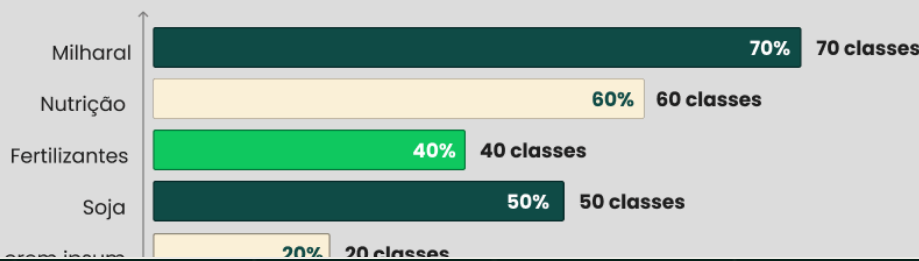


↓ Baixar gráfico



↓ Baixar gráfico

Relação de classes ▾



DASHBOARD GERAL, COM A VISÃO CONSOLIDADA DOS PROTOCOLOS

### MÓDULO 03 · ANÁLISE E DASHBOARDS

## A visão de todos os protocolos

- Distribuição de protocolos por **classe**, por **cultura**, por **cliente** e por **status**
- Contagens e percentuais vêm de um endpoint específico do back-end, buscados com **React Query**
- O cache mantém as visualizações atualizadas sem recarregamento manual da página
- No front-end, esses dados são mapeados para os mesmos vetores de rótulos e valores dos gráficos individuais

# Muitos gráficos na mesma tela custam caro

## SINTOMA

### Lentidão perceptível

A renderização simultânea de múltiplos gráficos com grandes conjuntos de dados travava a interface.

## CAUSA

### Tudo renderizado de uma vez

Gráficos fora da área visível eram montados junto com os que o usuário estava efetivamente olhando.

## SOLUÇÃO

### Carregamento sob demanda

Com **lazy loading**, os gráficos passaram a ser renderizados apenas quando o usuário navega até a seção correspondente.



05

# Considerações Finais

Avaliação do que foi entregue, limitações honestas, trabalhos futuros e o que o estágio deixou na formação.



# O objetivo central foi atendido no plano técnico

Substituir o registro descentralizado em planilhas por uma plataforma própria, com acesso controlado e análise automatizada.

## ENTREGA 01

### Centralização

O cadastro completo dos protocolos, com formulários extensos, validação declarativa e estruturas dinâmicas para as avaliações de campo.

## ENTREGA 02

### Controle de acesso

Cada colaborador interage apenas com as funcionalidades do seu papel, com restrição no nível das rotas e renovação transparente de sessão.

## ENTREGA 03

### Análise automatizada

Gráficos e dashboards gerados a partir dos dados consolidados e exportáveis como imagens padronizadas pela identidade visual da empresa.

# Dificuldades, limitação e trabalhos futuros

## DIFICULDADES SUPERADAS

### Regras de negócio e performance

- Traduzir o domínio agrônômico para a lógica dos formulários, com validação contínua junto ao cliente
- Manter desempenho adequado na renderização de múltiplos gráficos, mitigada com carregamento sob demanda

## LIMITAÇÃO OBSERVADA

### Adoção inicial reduzida

- A adoção pelo cliente foi inicialmente baixa
- Isso é atribuível à **mudança de cultura** exigida pela transição das planilhas para um sistema estruturado, e não a uma deficiência técnica da solução

## TRABALHOS FUTUROS

### O que viria em seguida

- Acompanhar a adoção ao longo do tempo
- Realizar testes de usabilidade com os colaboradores
- Evoluir os dashboards com novos tipos de análise

# A primeira experiência com software de verdade

Produto real, equipe estruturada, cliente externo e regras de negócio que ninguém tinha escrito.

## PLANO TÉCNICO

### Competências consolidadas

React e TypeScript, estado global com Zustand e Immer, formulários com Formik e Yup, integração com Axios e React Query e, sobretudo, **visualização de dados** com ApexCharts.

## PLANO INTERPESSOAL

### Comunicação e trabalho em equipe

Levantar requisitos e traduzir necessidades de negócio em solução técnica exigiu diálogo constante com o cliente e com os demais membros do time.

## DIREÇÃO DE CARREIRA

### Interfaces orientadas a dados

A área de visualização de dados passou a orientar os interesses profissionais, integrando construção de aplicações web com análise de informação.

# O que a graduação deu, e o que faltou

## DISCIPLINAS QUE SUSTENTARAM O TRABALHO

- **Engenharia de Software:** ciclo de desenvolvimento, requisitos e métodos ágeis
- **Programação Orientada a Objetos:** modularidade e responsabilidade única traduzidas na componentização do React
- **Banco de Dados:** entender as estruturas devolvidas pela API
- **Interação Humano-Computador:** decisões de usabilidade e organização das telas
- **Algoritmos e Estruturas de Dados:** manipulação de vetores e estruturas aninhadas

## LACUNAS PERCEBIDAS

- **Frameworks modernos de front-end**, como React e o ecossistema em torno do TypeScript, vistos de forma superficial nas obrigatórias
- **Testes automatizados de interface**, pouco trabalhados no curso e cada vez mais valorizados no mercado
- **Integração contínua, versionamento em equipe** (com pull requests e revisão de código) e deploy em nuvem

Cobrir essas lacunas teria reduzido a curva de aprendizado dos primeiros dias de estágio.

— OBRIGADO PELA ATENÇÃO

# Perguntas?

AUTOR

Lucas Henrique Lopes Costa

ORIENTADOR

Prof. Paulo Afonso Parreira Júnior

INSTITUIÇÃO

UFLA · Sistemas de Informação · 2026